Algorithm 1 Calculation of Kinematic Metrics and NJS

Input : Trajectory data $x(t), y(t)$ for each body part, sampling rate fps

Output : Trajectory length, acceleration, jerk and NJS for each body part

Compute time step : $dt \leftarrow \frac{1}{fps}$

Clean frames using : $likelihood > 0.90$ and non-missing x, y coordinates

Compute elbow angle :

```
df_clean['Elbow_angle'] = np.degrees( np.arctan2(df_clean['Paw_y']
df_clean['Elbow_y'], df_clean['Paw_x'] - df_clean['Elbow_x']) -
            np.arctan2(df_clean['Shoulder_y'] -df_clean['Elbow_y'],
            df_clean['Shoulder_x'] - df_clean['Elbow_x']))
```

$$\theta \leftarrow degrees \begin{pmatrix} \arctan 2(Paw_y - Elbow_y,\ Paw_x - Elbow_x) \\ -\arctan 2(Shoulder_y - Elbow_y,\ Shoulder_x - Elbow_x) \end{pmatrix}$$

for each body part bp **do**

Compute trajectory length :

```
def calc_trajectory_length(x, y):
    mask = ~np.isnan(x) & ~np.isnan(y)
    dx, dy = np.diff(x[mask]), np.diff(y[mask])
    return np.sum(np.sqrt(dx**2 + dy**2))
```

$$length_{bp} \leftarrow \sum \sqrt{(\Delta x)^2 + (\Delta y)^2}$$

Compute velocity :
$v_x \leftarrow \frac{\Delta x}{dt}$, $v_y \leftarrow \frac{\Delta y}{dt}$

```
df_clean[f"{bp}_vx"]df_clean[f"{bp}_x"].diff() / dt
df_clean[f"{bp}_vy"] = df_clean[f"{bp}_y"].diff() / dt
df_clean[f"{bp}_speed"] = np.sqrt(df_clean[f"{bp}_vx"]**2
 + df_clean[f"{bp}_vy"]**2)
```

Compute acceleration : $a_x \leftarrow \frac{\Delta v_x}{dt}$, $a_y \leftarrow \frac{\Delta v_y}{dt}$

```
df_clean[f"{bp}_ax"] = df_clean[f"{bp}_vx"].diff() / dt
df_clean[f"{bp}_ay"] = df_clean[f"{bp}_vy"].diff() / dt
df_clean[f"{bp}_accel"] = np.sqrt(df_clean[f"{bp}_ax"]**2
 + df_clean[f"{bp}_ay"]**2)
```

Compute acceleration magnitude : $accel \leftarrow \sqrt{a_x^2 + a_y^2}$

Compute jerk : $j_x \leftarrow \frac{\Delta a_x}{dt}$, $j_y \leftarrow \frac{\Delta a_y}{dt}$

```
df_clean[f"{bp}_jerk_x"] = df_clean[f"{bp}_ax"].diff() / dt
df_clean[f"{bp}_jerk_y"] = df_clean[f"{bp}_ay"].diff() / dt
df_clean[f"{bp}_jerk"] = np.sqrt(df_clean[f"{bp}_jerk_x"]**2
 + df_clean[f"{bp}_jerk_y"]**2)
```

Compute Normalized Jerk Score :

```
def calculate_njs(jerk, time, duration, length):
    integral_jerk_squared = np.trapz(jerk**2, time)
    njs = np.sqrt(0.5 * integral_jerk_squared * (duration**5) / (length**2))
    return njs\\ \\
```

$$NJS_{bp} \leftarrow \sqrt{0.5 \times I_{jerk} \times \frac{duration^5}{length_{bp}^2}}$$

end for

Algorithm 2 Synergy Analysis Between Paw and Elbow

Input : Paw trajectory and elbow angle $\theta(t)$

Output : Global and phase-specific synergy percentages

Compute vertical paw displacement : $\Delta y_{paw} \leftarrow diff(Paw_y)$

```
delta_paw_y = df_clean["Paw_y"].diff()
```

Compute horizontal paw displacement : $\Delta x_{paw} \leftarrow diff(Paw_x)$

```
delta_paw_x = df_clean["Paw_x"].diff()
```

Compute elbow angle variation : $\Delta\theta \leftarrow diff(Elbow\_angle)$

```
delta_angle = df_clean["Elbow_angle"].diff()
```

Compute global synergy vector :

```
synergy = (np.sign(delta_paw_y) == np.sign(delta_angle)).astype(int)
```

$$synergy \leftarrow (\text{sign}(\Delta y_{paw}) = \text{sign}(\Delta\theta))$$

Compute global synergy percentage :

```
synergy_percent = 100 * synergy.sum() / len(synergy.dropna())
 if len(synergy.dropna()) > 0 else np.nan
```

$$Synergy_{global} \leftarrow 100 \times \frac{\sum synergy}{N_{valid}}$$

Define phase indexes :

```
direction_idx = df_clean[df_clean["delta_paw_x or y"] > < 0].index
```

$$advance \leftarrow \Delta x_{paw} > 0$$

$$lift \leftarrow \Delta y_{paw} < 0$$

$$drop \leftarrow \Delta y_{paw} > 0$$

for each phase $p \in \{advance, lift, drop\}$ **do**

Select valid frames : $valid \leftarrow indexes_p \cap valid\_frames$

Compute same-direction count :

```
        def compute_synergy(indexes):
valid = indexes.intersection(df_clean.dropna(subset=["delta_paw_y",
        "delta_angle"]).index) if len(valid) > 0:
same_dir = (np.sign(df_clean.loc[valid, "delta_paw_y"]) == np.sign(
         df\_clean.loc[valid, "delta_angle"])).sum()
         return 100 * same_dir / len(valid) else:  return np.nan
```

$$same\_dir \leftarrow \sum [\text{sign}(\Delta y_{paw}) = \text{sign}(\Delta\theta)]$$

Compute phase-specific synergy :

```
synergy_direction = compute_synergy(direction_idx)
```

$$Synergy_p \leftarrow 100 \times \frac{same\_dir}{N_p}$$

end for